DevOps Application Deployment

Clean End-to-End Project Documentation

React.js • Docker • Docker Compose • Bash • GitHub • Jenkins • Docker Hub • AWS EC2 • Nginx • Uptime Kuma

Project	Devops-build
GitHub	github.com/dhan8292/devops-build/tree/dev
Application	React.js / Create React App
Web Server	Nginx
CI/CD	Jenkins
Container Registry	Docker Hub
Cloud	AWS EC2
Monitoring	Uptime Kuma
Status	Completed

1. Project Overview

This project deploys a React application in a production-style environment. The React application is built into static files, packaged in a Docker image, and served by Nginx. Jenkins automates the Docker image build and push process. Docker Hub stores development and production images, AWS EC2 provides the server, and Uptime Kuma monitors application availability.

The documented implementation is the dhan8292/devops-build repository on the dev branch. The project does not contain a separate backend API, database, or server-side business-logic application.

2. Project Objectives

• Deploy the React application as a production web application.

• Containerize the application using Docker and Nginx.

• Use Docker Compose for repeatable container startup.

• Automate image creation with Bash and Jenkins.

• Use separate Docker Hub repositories for development and production images.

• Use Git branches to control image destinations.

• Run the application on AWS EC2 using HTTP port 80.

• Monitor application availability using Uptime Kuma.

3. Architecture

Deployment flow:

```
Developer → GitHub → Jenkins → Docker Build → Docker Hub → AWS EC2 → Docker Container → Nginx → Port 80 → User
```

The React production build is copied into an Nginx container. Jenkins builds and pushes the Docker image. The selected image is then run on the EC2 server. Nginx serves the static React files over HTTP port 80.

4. Application: Frontend and Backend

Frontend: React.js / Create React App. `npm start` is used for development and `npm run build` creates the production files in `build/`.

Backend: There is no separate backend API, database, or server-side application in this repository. Nginx is the production web server.

5. Repository Structure

Path	Purpose
src/	React source code
public/	Public assets
build/	Production React build
.dockerignore	Docker build exclusions
.gitignore	Git exclusions
Dockerfile	Nginx production image
docker-compose.yml	Container configuration
build.sh	Docker build script
deploy.sh	Branch-aware deployment script
Jenkinsfile	Jenkins CI/CD pipeline
README.md	Project documentation
screenshorts/	Evidence screenshots

6. Application Setup

Install dependencies:

```
npm install
```

Run the development application:

```
npm start
```

Create the production build:

```
npm run build
```

The production build creates the `build/` directory. Docker uses these generated files instead of the development server.

7. Dockerfile

The Dockerfile uses the lightweight `nginx:alpine` image, removes the default Nginx content, copies the React build into the Nginx web directory, and exposes port 80.

```
FROM nginx:alpine
RUN rm -rf /usr/share/nginx/html/*
COPY build/ /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Build the image:

```
docker build -t react-devops-app .
```

8. Docker Compose

The Docker Compose configuration defines the react-app service, builds from the project directory, uses the container name react-devops, and maps host port 80 to container port 80.

```
docker compose up -d --build
docker ps
```

9. Bash Automation

build.sh

```
#!/bin/bash
echo "Building Docker Image ..."
docker build -t react-devops-app .
echo "Build Completed"
```

deploy.sh

The deployment script detects the current Git branch, creates a build-number-style tag when Jenkins BUILD_NUMBER is unavailable, builds the image, authenticates to Docker Hub, pushes the image to the correct repository, and replaces the running container.

Branch mapping:

```
dev → dhanu92/react-devops-dev
master → dhanu92/react-devops-prod
```

The deployed container uses port mapping 80:80 and the name react-container. The source deploy.sh uses docker login with -p; for production, --password-stdin with Jenkins Credentials is preferred. The Jenkinsfile uses the safer --password-stdin approach.

10. Git and GitHub

The documented implementation uses the dev branch. Typical CLI commands are:

```
git checkout dev
git add .
git commit -m "deployment changes"
git push origin dev
```

The intended promotion model is dev for development images and master for production images.

11. Docker Hub

Development repository: dhanu92/react-devops-dev
Production repository: dhanu92/react-devops-prod

The Jenkinsfile selects the repository based on the branch and uses BUILD_NUMBER for image tags. The assignment requires the production repository to be private and the development repository to be public. The source documentation confirms the repository names but does not independently prove their current privacy

settings.

12. Jenkins CI/CD Pipeline

Stage	Purpose
Checkout	Checks out the source using checkout scm
Detect Branch	Reads BRANCH_NAME
Deployment Info	Shows the dev/master decision
Build Docker Image	Runs docker build
Push to DockerHub	Logs in, tags, and pushes the image
Post	Reports success or failure

Jenkins credential ID: dockerhub-creds. Docker Hub authentication uses --password-stdin.

13. AWS EC2 Deployment

The assignment specifies a t2.micro EC2 instance. The server hosts Docker and the production React/Nginx container.

```
AWS EC2 → Docker Engine → react-container → Nginx :80 → React build files
```

HTTP port 80 is the application endpoint. Jenkins and monitoring use separate management ports as documented by the project.

14. AWS Security Group

Port	Purpose	Recommended Source
22/TCP	SSH	Developer public IP /32 only
80/TCP	Application HTTP	0.0.0.0/0 when public access is required
8080/TCP	Jenkins	Trusted IPs only where possible
3001/TCP	Uptime Kuma UI	Trusted IPs only where possible

SSH should not be open to the entire internet. Jenkins and Uptime Kuma should also be protected where possible.

15. Monitoring with Uptime Kuma

Uptime Kuma is the open-source monitoring component used by the project. It can monitor the deployed HTTP endpoint and show application availability. The project documents port 3001 for its interface. An HTTP monitor can be configured to check the deployed application URL and send notifications when the application goes down.

16. End-to-End Deployment Flow

1. Developer changes the React source code.
2. Code is committed and pushed to GitHub.
3. Jenkins is triggered for the configured branch.
4. Jenkins checks out the repository and detects dev or master.
5. Jenkins builds the Docker image.

6. dev branch → image is pushed to dhanu92/react-devops-dev.

7. master branch → image is pushed to dhanu92/react-devops-prod.

8. The deployment runs the application image on AWS EC2.

9. Docker maps host port 80 to Nginx port 80.

10. Users access the application over HTTP.

11. Uptime Kuma monitors application availability.

17. Common Challenges and Solutions

Challenge	Solution
React production build missing	Run npm run build and verify build/ before the Docker build.
Port 80 already in use	Stop/remove the previous application container before starting the new one.
Docker Hub authentication	Configure dockerhub-creds in Jenkins and use --password-stdin.
Wrong dev/prod repository	Use branch detection and explicit repository variables.
Image version tracking	Use Jenkins BUILD_NUMBER as the Docker image tag.
Remote deployment reliability	Use a repeatable deployment script and recreate the named container.
Application availability	Configure Uptime Kuma HTTP monitoring and notifications.

Useful commands:

```
docker ps
docker images
docker logs react-container
docker stop react-container
docker rm react-container
git status
git branch
curl http://localhost:80
```

18. Security Practices

• Use Jenkins Credentials for Docker Hub secrets.

• Use --password-stdin for Docker login.

• Keep the production Docker Hub repository private.

• Restrict SSH port 22 to the developer IP.

• Restrict Jenkins and monitoring ports to trusted sources.

• Never commit passwords, tokens, cloud keys, or private keys.

• Use build-number image tags so builds are traceable.

19. Verification Checklist

■ GitHub dev branch contains source and DevOps files.

■ npm run build creates build/.

■ Docker image builds successfully.

■ Docker container is running.

■ HTTP port 80 serves the application.

■ Docker Compose starts the service.

■ build.sh completes successfully.

■ deploy.sh selects the correct dev/master repository.

■ Jenkins pipeline succeeds.

■ Dev image is pushed to the dev repository.

■ Master image is pushed to the production repository.

■ EC2 Security Group allows HTTP and restricts SSH.

■ Deployed application is reachable.

■ Uptime Kuma reports healthy status.

20. Submission Evidence

Evidence	Purpose
GitHub repository	Source code and DevOps configuration
Jenkins login/config	Jenkins setup
Jenkins execution	Build and push result
AWS EC2 console	Cloud server
Security Group	Network access rules
Docker Hub dev	Development image and tags
Docker Hub prod	Production image and tags
Deployed site	Application proof
Uptime Kuma	Health monitoring proof

The repository contains screenshots dated March 16–20, 2026. Each screenshot should be matched to the relevant submission requirement before submission.

21. Interview Explanation

“I deployed a React application on AWS EC2 using Docker and Nginx, with the application exposed on HTTP port 80. I created Jenkins CI/CD automation to build Docker images and push them to separate development and production Docker Hub repositories based on the Git branch. I also created Bash automation scripts and used Uptime Kuma to monitor application availability.”

Frontend: React.js. It provides the user interface and is compiled into static production files.
Backend: There is no separate backend/API/database in this project. Nginx serves the React frontend.
Main practical challenges: Docker builds, port conflicts, Docker Hub authentication, branch-based dev/prod handling, and reliable container replacement.

22. Resume Project Description

DevOps Application Deployment Capstone | Docker, Jenkins, AWS EC2, Docker Hub, GitHub, Nginx, Bash, Uptime Kuma

• Containerized and deployed a React.js application using Docker and Nginx on AWS EC2 with HTTP port 80.

• Built Jenkins CI/CD automation with branch-based Docker image promotion to separate development and production Docker Hub repositories.

• Automated image build and deployment using Bash scripts and implemented application availability monitoring with Uptime Kuma.

23. Conclusion

This project demonstrates a practical DevOps workflow from source control to production deployment. The React application is built for production, packaged with Docker, served through Nginx, published to Docker Hub, and managed through Jenkins automation. AWS EC2 provides the runtime environment and Uptime Kuma provides application health visibility.

The project demonstrates hands-on work with Git/GitHub, Linux, Bash, Docker, Docker Compose, Nginx, Jenkins, Docker Hub, AWS EC2, Security Groups, and monitoring.

Source note: This document was recreated from the uploaded project documentation. Claims about external infrastructure state, Docker Hub privacy, and current monitoring status should be supported by the corresponding project screenshots when used as submission evidence.